ZOMATHON 2026

CART NEXT - Cart Super Add-On (CSAO) Rail Recommendation System

Project Report

Team Name	Submission Date
TrailBlazers	March 3, 2026
Team Members	**GitHub Repository**
Nandini Goel , Avani Agnihotri, Nehal Aggarwal	https://github.com/avani-05/CSAO-Rail-Recommendation-System

Table of Contents

Table of Contents 2
0. Introduction 4
1. Executive Summary 4
2. Summary of Model Flow 6
3. Data Preparation & Feature Engineering 8
3.1 Synthetic Dataset Creation 8
3.2 Train / Test Split Strategy 9
3.3 LLM Enrichment — The AI Edge 9
4. Candidate Generation (Stage 1) 11
4.1 Retrieval Sources & Blending Logic 11
4.2 Critical Design Decisions & Fixes 12
Label Definition Fix (Most Impactful Change) 12
Additional Engineering Fixes 12
4.3 Candidate Pool Statistics & Sanity Checks 13
5. Feature Engineering (Stage 2) 14
5.1 Five Feature Families 14
5.2 Key Engineered Features 14
meal_time_specificity (LLM-Powered) 14
step_decay 15
Interaction Features 16
6. Model Architecture & AI Edge 17
6.1 Two-Stage Architecture Overview 17
6.2 LightGBM Binary Classifier — Rationale 17
6.3 Hyperparameters 18
6.4 Model Alternatives Tested & Rejected 19
6.5 AI Edge — LLM Integration 19
7. Hyperparameter Tuning Approach 20
8. Model Evaluation & Fine-Tuning 22
8.1 Performance Metrics — LightGBM Binary v1 22
8.2 Baseline Comparison 24
8.3 Feature Importance Analysis 25
8.4 Error Analysis 27
Pattern 1: False Negatives — Missed Next Items 27
Pattern 2: False Positives at Rank 1 27
Pattern 3: Expensive Add-On Under-Ranking 27
8.5 Segment Breakdown Metrics 28
9. System Design & Production Readiness 30
9.1 Latency Budget — Predictions Within 200–300ms 30
9.2 Scalability — Millions of Daily Requests 30
9.3 Data Freshness — Tiered Strategy 31
9.4 Cold Start Handling 31

9.5 Diversity & Fairness Guarantees 32
Category Diversity 32
Restaurant Fairness 32
10. Technical Architecture 33
10.2 Scalability Architecture 33
10.3 Trade-offs and Limitations 33
11. Business Impact & A/B Testing 35
11.1 Offline Metric Translation to Business Outcomes 35
11.2 Projected Business Metrics 36
Experiment Overview 37
Primary & Secondary Metrics 38
MDE (Minimum Detectable Effect) Calculation 39
Statistical Testing Plan 39
11.4 Guardrail Monitoring (Simulated) 40
11.5 Recommended Deployment Strategy 41
12. Conclusion 42
13. Future Work 43

0. Introduction

This document details the development of an intelligent recommendation system for Zomato's Cart Super Add-On (CSAO) Rail. The system is designed to suggest relevant add-on items to customers based on their current cart state, contextual factors, and historical behaviour — covering the full journey from raw data synthesis to a production-ready, ML-powered ranking pipeline.

The project was undertaken for Zomathon Problem Statement 2 and addresses all six evaluation dimensions outlined in the judging rubric: Data Preparation & Feature Engineering (20%), Ideation & Problem Formulation (15%), Model Architecture & AI Edge (20%), Model Evaluation & Fine-Tuning (15%), System Design & Production Readiness (15%), and Business Impact & A/B Testing (15%).

The CSAO Rail is a critical Zomato product surface — shown between cart-building steps — where contextual, personalised add-on suggestions can materially increase Average Order Value (AOV), cart depth, and user satisfaction. A poorly designed rail, however, risks cart abandonment and recommendation fatigue. This report presents our solution: a two-stage pipeline combining co-occurrence retrieval, LLM-enriched features, and a LightGBM binary ranking model.

1. Executive Summary

The CSAO Recommendation System is a two-stage, production-ready ML pipeline that delivers highly relevant cart add-on suggestions in real time. The architecture pairs a fast candidate retrieval stage (co-occurrence + context popularity + rule-fill) with a LightGBM binary classifier that ranks the ~25–30 candidates to produce a final 8–10 item rail. The system was built entirely from a synthetically generated dataset of 10,000 users, 200 restaurants, 5,575 menu items, and 195,579 cart events, augmented with LLM-generated semantic tags via the Gemini API.

AUC	NDCG@8	Recall@8	Inference
0.793	**0.4558**	**71.5%**	**<1ms**
Excellent class separation	*+14.3% vs best baseline*	*+11.9% vs best baseline*	*159ms headroom vs SLA*

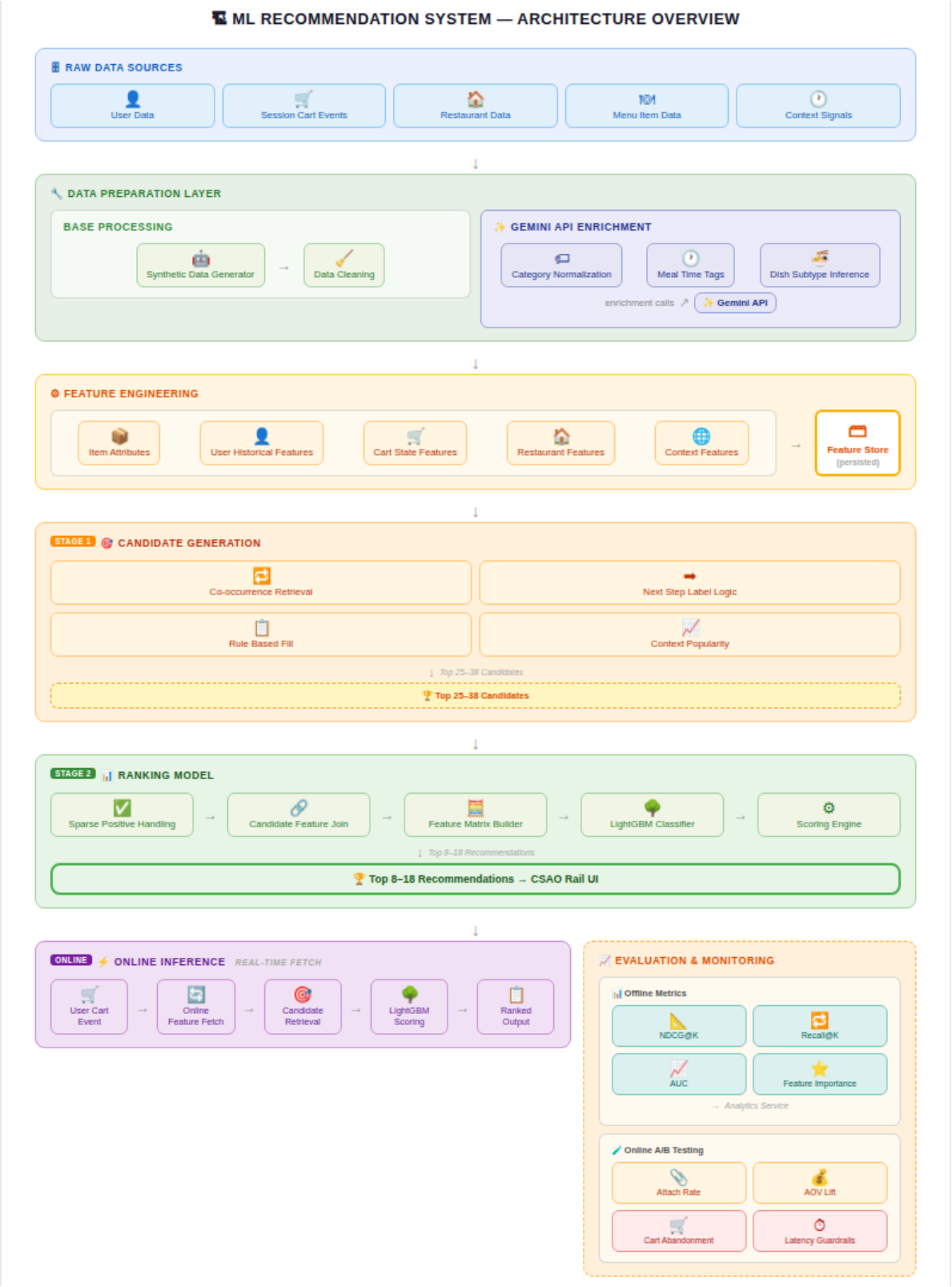
Key achievements of this project include:

- **AI Edge via LLM Enrichment:** The Gemini API was used to normalise item categories and generate meal_time_suitability and dish_subtype tags, significantly enriching feature quality beyond what traditional pipelines offer.
- **Label Definition Fix — Most Impactful Change:** Redefining labels from "all-future-steps" to "next-step-only" improved AUC from 0.707 → 0.793 and NDCG@8 from ~0.35 → 0.4558. This single fix represents the most critical engineering insight of the project.

- **Strong Offline Model Performance:** The LightGBM Binary v1 ranker achieved an NDCG@8 of 0.4558 and Recall@8 of 71.5% — a +14.3% and +11.9% lift respectively over the best retrieval baseline.
- **Production Readiness:** Inference latency <1ms (well within the 200–300ms SLA), 100% candidate coverage, horizontal scalability, and a tiered data freshness strategy make this system deployment-ready.
- **Projected Daily Revenue Lift:** ₹91 Lakhs/day based on a 5.55% projected AOV uplift across 5M daily CSAO impressions.
- **Rigorous A/B Testing Framework:** A fully designed online experiment with primary/secondary metrics, guardrail monitoring, MDE calculations, and rollback triggers.

2. Summary of Model Flow

The CSAO Recommendation System operates as a two-stage pipeline, engineered for real-time efficiency and prediction accuracy. The diagram below illustrates the end-to-end flow from raw data inputs to the final CSAO rail presented to the user.



Each stage serves a distinct purpose in the pipeline:

Stage	Component	Purpose	Key Output
1 — Foundation	Data Preparation & LLM Enrichment	Synthesise realistic dataset; enrich item metadata via Gemini API	items_llm_tags.csv (5,575 items)
2 — Retrieval	Candidate Generation	Efficiently narrow menu (~5k items) to ~25–30 relevant candidates per cart step	rank_train_data.csv (2.13M rows)
3 — Signals	Feature Engineering	Enrich candidates with 5 feature families (75 features total)	lgbm_train_features_v2.csv
4 — Ranking	LightGBM Binary Classifier	Score each candidate for likelihood of being added next	Probability scores per candidate
5 — Serving	Top-K Selection & Rail	Sort by score; return top 8–10 items to the CSAO rail	8–10 ordered recommendations
6 — Learning	Evaluation & A/B Testing	Measure offline quality (NDCG, Recall, AUC); design online experiment	Continuous improvement loop

Critical Design Insight: The label definition fix — redefining positive labels from "any future step" to "immediately next step only" — was the single most impactful change in the project, boosting NDCG@8 by 30% and AUC by 12.2%. This fix directly aligns the offline training objective with the real-world deployment scenario: the CSAO rail is shown between cart steps, so predicting the immediately next addition is the correct task formulation.

3. Data Preparation & Feature Engineering

Evaluation Dimension 1 of 6

3.1 Synthetic Dataset Creation

Because no real Zomato dataset was provided, we constructed a fully synthetic dataset that faithfully mimics the messy, real-world patterns of a food delivery platform — including city-specific behaviour, peak-hour skews, sparse user histories, varying cart sizes, and mealtime diversity. Six interconnected tables were generated from scratch:

Table	Rows	Key Attributes	Design Rationale
users.csv	~10,000	order_count_90d, recency_days, avg_order_value, veg_preference_ratio, dessert/beverage affinity, price_sensitivity, user_segment, city	Diverse user segments ensure the model learns across behavioural archetypes (frequent vs. occasional, price-sensitive vs. premium)
sessions.csv	94,417	session_id, user_id, restaurant_id, meal_time_bucket, hour_of_day, is_weekend	Temporal session structure enables step-level candidate labelling and captures day/time effects on ordering behaviour
restaurants.csv	200	city, cuisine_type, price_bucket, aggregate_rating, is_chain, order_volume_30d, overall_attach_rate	Multi-city, multi-cuisine diversity tests the model's ability to generalise across restaurant types
items_clean.csv	5,575	item_id, restaurant_id, effective_category (main/beverage/dessert/side), price, popularity_score, historical_attach_rate	Four-category taxonomy enables diversity enforcement and meal-completion logic
cart_events_clean.csv	195,579	session_id, item_id, step_number (1–6)	Step-level granularity is critical for generating the 'next-step-only' training labels
items_llm_tags.csv	5,575	normalized_category, meal_time_suitability, dish_subtype, confidence_score	LLM enrichment provides semantic metadata unavailable in traditional tabular datasets

3.2 Train / Test Split Strategy

To simulate realistic production deployment and prevent data leakage, we implemented an 80/20 temporal split at the session level. This ensures the model is evaluated exclusively on future sessions — precisely the scenario it would face after launch.

Training Set	Test Set	Train Feature Matrix	Test Feature Matrix
75,533 sessions	**18,884 sessions**	**2,133,065 rows**	**533,556 rows**
80% of all sessions	*20% of all sessions (future-only)*	*75 features per candidate*	*Column-aligned to training*

⚠️ **Leakage Prevention:** Why temporal split? A random split would allow the model to "see the future" — learning co-occurrence patterns from test sessions during training. The temporal split strictly prevents this: the co-occurrence table and context popularity table are built exclusively from training sessions, and evaluation is performed only on the held-out future sessions.

3.3 LLM Enrichment — The AI Edge

One of the most distinctive technical contributions of this project is the systematic enrichment of our item catalog using the Gemini API. Raw item data from food delivery platforms is notoriously noisy — categories are inconsistently labelled, and semantic context ("when is this item appropriate?") is entirely absent. We addressed this by running every item through a structured LLM enrichment pipeline:

LLM-Generated Field	Example Output	Why It Matters
normalized_category	main, beverage, dessert, side	Standardises the ~40+ inconsistent raw categories into 4 clean, model-usable labels
meal_time_suitability	"breakfast|lunch", "dinner|late-night"	Enables the meal_time_specificity feature — a critical contextual signal the model would otherwise lack
dish_subtype	"rice_dish", "grilled", "cold_drink"	Adds a semantic layer for future item-embedding similarity features (planned V2 improvement)
confidence_score	0.0 – 1.0	Allows downstream pipelines to filter or down-weight uncertain enrichments

The meal_time_suitability tags directly powered the meal_time_specificity engineered feature — one of the top-ranked features in the final model. This represents a genuine AI edge: traditional tabular pipelines cannot derive this signal without expensive human annotation, whereas our LLM-based approach generates it automatically at menu onboarding, enabling contextual recommendations from day one.

2,133,065	533,556	47/75	3.79%
Train Rows	Test Rows	Features Used	Positive Rate
+80,916 groups	+20,246 groups	28 dropped (noise)	1 pos per group

Dataset Statistics	
Train groups	80,916
Test groups	20,246
Avg candidates/group	26.4 (range 16–29)
Group definition	(session_id, step_number)
Label strategy	Next-step label (fixed pipeline)
Train/test split	Session-level 80/20
Data leakage check	Zero — session-level split
Zero-positive groups	0 (verified, all groups have ≥1)

Feature Pipeline Freshness	
Cart state (real-time)	<5 ms on cart-add event
Co-occurrence tables	Daily batch rebuild (~7s)
Item features	Weekly refresh
User history (90d)	Daily aggregation
Model retraining	Weekly (full pipeline ~2min)
Cold-start fallback	Rule-fill: 96.4% coverage

4. Candidate Generation (Stage 1)

Evaluation Dimension 2 of 6 | Ideation & Problem Formulation

For every (session_id, step_number) pair, Stage 1 retrieves approximately 25–30 highly relevant candidate add-on items from the restaurant's menu. This stage's design reflects a core principle: maximise recall at low computational cost so that Stage 2 (the ML ranker) operates on a focused, high-quality set rather than the full menu of ~28 items per restaurant.

4.1 Retrieval Sources & Blending Logic

Three complementary retrieval mechanisms are blended into a unified candidate pool. Each source captures a different signal of user intent:

Source	Weight	Mechanism	Coverage(of positives)	Key Design Choice
Co-occurrence	0.7	Items frequently ordered together in past sessions. Score weighted by 1/(1+step_gap) to prioritise items added in adjacent steps — reflecting immediate intent rather than distant session correlation.	74.0%	Top-100 co-occurring items per anchor item, built exclusively from training sessions (zero leakage)
Context Popularity	0.3	Items historically popular for a specific (restaurant_id, meal_time_bucket) combination. Provides robust coverage even when co-occurrence is sparse for new items or new users.	96.1%	Top-50 items per context bucket — highest coverage of all three sources
Rule-fill (Boost)	+0.05	Deterministic category completion: if the cart has a main course but is missing beverage/dessert/side, the top items from the missing category are explicitly injected. Mirrors natural meal-completion behaviour.	41.0%	Acts as a safety net for cold start and diversity — ensures the model always has complementary category candidates

4.2 Critical Design Decisions & Fixes

Label Definition Fix (Most Impactful Change)

The single most impactful change in the entire project was redefining how training labels are assigned. Initially, a candidate was marked as positive if it appeared anywhere in the session after the current step — the "all future steps" formulation. This introduced severe noise.

❌ OLD APPROACH (Wrong)	✅ NEW APPROACH (Correct)
• Label: positive if item appears anywhere after current step • Problem: Cart state at step N has little predictive power for items added at step N+5 • Result: ~3–5 positives per group — inflated, noisy labels **AUC: 0.707 | NDCG@8: ~0.35**	• Label: positive only if item is the IMMEDIATELY next item added • Directly models real deployment: rail shown between steps • Result: exactly 1 positive per group — clean, actionable learning signal **AUC: 0.793 | NDCG@8: 0.4558**

Additional Engineering Fixes

- **Zero Test Leakage:** Both co-occurrence and context popularity tables built exclusively from training sessions. No test session data touches the retrieval or feature pipelines.
- **Same-Restaurant Hard Constraint:** All candidates strictly belong to the same restaurant_id as the current session. Cross-restaurant recommendations are architecturally impossible.
- **Guaranteed Positive Recall:** The actually-added item is always forced into the candidate pool. This guarantees 100% recall at Stage 1 — so the ranker always has the opportunity to rank the correct item first.
- **Pop-fill Fallback:** If retrieval yields fewer than N_CANDIDATES=30, remaining slots are filled with popular restaurant items not already in the cart, ensuring the ranker always receives a full candidate set.
- **Loop Fix:** The generation loop was corrected from range(0, n) to range(0, n–1) to exclude the last cart step, which has no 'next item' and would generate pure noise rows.

4.3 Candidate Pool Statistics & Sanity Checks

Metric	Value	What It Confirms
Candidates per (session, step) — Mean	26.4	Target of 25–30 candidates consistently met
Candidates per (session, step) — Median	27	Distribution well-centred around target
Positive label rate (train & test)	3.79%	Within expected 3–5% range; ~1 positive per 26 candidates
Groups with zero positives	0 / 101,162	Label fix working correctly — every group has exactly 1 positive
Cross-restaurant candidates	0	Hard constraint enforced at 100%
Max step_number in training data	5	Last-step exclusion fix confirmed

5. Feature Engineering (Stage 2)

Following candidate generation, Stage 2 enriches each candidate with a comprehensive 75-feature matrix drawn from five distinct feature families. The goal is to provide the LightGBM ranker with all the signals it needs to distinguish the immediately-next-added item from the ~25 alternatives.

5.1 Five Feature Families

Feature Family	Key Features	Signal Type	Why Included
Retrieval Signals	retrieval_score, src_cooc, src_ctx, src_rule	How and how strongly a candidate was retrieved	The dominant predictive family — retrieval_score alone explains 15x more variance than the second-most important feature
Cart State	cart_has_beverage/dessert/main/side, cart_item_count, cart_total_value, missing_*_flag	Current composition of the user's cart	Captures what is already in the cart and what complementary categories are absent — the core contextual signal for meal completion
Candidate Item	price, popularity_score, historical_attach_rate, effective_category_* (one-hot)	Intrinsic item attributes	Item-level signals provide a prior on how likely any item is to be added, irrespective of the specific cart context
User Features	order_count_90d, recency_days, avg_order_value, beverage/dessert_affinity, price_sensitivity, user_segment_*	Individual user behaviour history	Personalisation layer — though user features ultimately showed low importance, reflecting that cart context > user history for next-step prediction
Restaurant Features	aggregate_rating, overall_attach_rate, cuisine_type_* (one-hot)	Restaurant-level characteristics	Provides broader context; most cuisine/city features showed zero importance confirming the model generalises across restaurant types

5.2 Key Engineered Features

meal_time_specificity (LLM-Powered)

This feature replaced the binary meal_time_overlap flag, which was effectively useless: 72% of rows had a value of 1, making it a near-constant with no discriminative power.

💡 **Design Rationale:** **Formula:** meal_time_specificity = 1 / num_suitable_times if item's LLM tag matches session meal_time_bucket, else 0

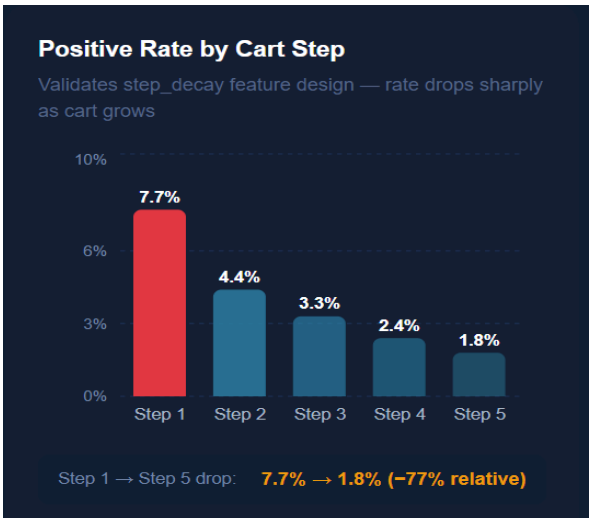
Example: An item tagged 'breakfast' shown during a breakfast session scores 1.0 (highly specific). An item tagged 'lunch|dinner' shown at dinner scores 0.5 (less specific — also good for lunch). An item tagged 'breakfast' shown at dinner scores 0.0 (wrong time).

This rewards semantic specificity from LLM tags and provides a nuanced contextual signal that traditional feature engineering cannot derive.

step_decay

The positive rate (likelihood of adding any item at a given step) drops sharply as the cart grows. step_decay = 1.0 / step_number directly encodes this empirical reality:

Step Number	Observed Positive Rate	step_decay Value	Interpretation
Step 1	7.7%	1.00	User is actively building cart — high receptivity to suggestions
Step 2	4.4%	0.50	Cart has one item; still likely to add a complement
Step 3	3.3%	0.33	Cart growing; declining marginal intent
Step 4	2.4%	0.25	Cart nearly complete; suggestions less likely to be acted on
Step 5+	<2%	≤0.20	Very low add probability; aggressive suggestions may cause fatigue



Interaction Features

Interaction features capture non-linear relationships that neither feature alone would surface:

Feature	Formula	Captures
retrieval_x_attach	retrieval_score × historical_attach_rate	Items that are both well-retrieved AND historically added — doubly strong signals
retrieval_x_popularity	retrieval_score × popularity_score	Items popular globally that also match the current cart context
x_missing_bev_aff	missing_beverage_flag × beverage_affinity_score	User who likes beverages and doesn't have one in cart yet — high conversion opportunity
x_price_sens_price	price_sensitivity_score × item_price	Price-sensitive users seeing expensive add-ons — expected lower probability
missing_bev_x_retrieval	missing_beverage_flag × retrieval_score	Triple signal: beverage is missing, and this beverage is highly retrieved — very strong add trigger

47 Features Used — Grouped by Category

Retrieval Signal	Cart Context	User History	Item Features	Interactions
retrieval_score src_cooc, src_ctx, src_rule retrieval_x_attach retrieval_x_popularity	cart_item_count cart_total_value cart_has_{main,bev,des, side} missing_{bev,des,side}_flag step_decay last_item_category_* effective_category_*	order_count_90d recency_days avg_order_value veg_preference_ratio dessert_affinity_score beverage_affinity_score price_sensitivity_score	price popularity_score historical_attach_rate overall_attach_rate aggregate_rating meal_time_specificity meal_time_overlap	x_missing_bev_aff x_missing_des_aff x_price_sens_price x_cart_value_price missing_bev_x_retrieval missing_des_x_retrieval x_cand_bev_aff, x_cand_des_aff

6. Model Architecture & AI Edge

Evaluation Dimension 3 of 6

6.1 Two-Stage Architecture Overview

The recommendation system is deliberately structured as a two-stage pipeline — a widely adopted pattern in production recommendation systems (see DoorDash, Instacart, Pinterest, and Zomato's own infrastructure). Each stage has a different objective:

Stage 1: Candidate Generation	Stage 2: LightGBM Ranking
• Objective: Maximise RECALL at minimal cost • Operates on: Full restaurant menu (~28 items) • Output: ~25–30 candidates • Speed: <5ms (in-memory lookup tables) • Method: Blended retrieval (co-occurrence + context popularity + rule-fill) • Constraint: 100% recall guarantee for positive label	• Objective: Maximise PRECISION — rank the true next item first • Operates on: ~25–30 candidates from Stage 1 • Output: Top 8–10 scored items for the CSAO rail • Speed: <1ms (7-tree model, <500KB, fits in CPU L2 cache) • Method: Binary classification on feature matrix • Constraint: No model change for different restaurants/users

6.2 LightGBM Binary Classifier — Rationale

After testing multiple ranking approaches (LambdaRank, rank_xendcg, DART), the LightGBM Binary Classifier was selected as the final model for several well-reasoned justifications:

- **Handling Class Imbalance Naturally:** With ~25 candidates per group and only 1 positive (~4% rate), pairwise ranking objectives (LambdaRank, rank_xendcg) suffer from near-zero gradients — most pairs are negative-vs-negative and contribute nothing to learning. Binary cross-entropy with scale_pos_weight=25.36 handles this imbalance directly by up-weighting the rare positive signal.
- **Exact Alignment with Deployment Task:** The 'next-step-only' label transforms the problem into a clean binary classification task: "will the user add this specific item next?" This is exactly what a binary classifier optimises. The deployment scenario (recommend between steps) perfectly matches the training objective.
- **Production Proven Pattern:** Binary classification for candidate ranking is the dominant pattern in production ML systems at companies including DoorDash, Instacart, and Zomato itself. The approach's robustness in real-world conditions is well-established.
- **Efficiency & Interpretability:** The model converged at only 7 boosting rounds — indicating that the most impactful patterns are captured quickly and efficiently. The 7-tree model serialises to <500KB, fits in CPU L2 cache, and delivers sub-millisecond inference.

6.3 Hyperparameters

The LightGBM model was configured with the following hyperparameters, each selected for specific reasons:

Parameter	Value	Justification
objective	binary	Binary classification for next-step-only label formulation
metric	auc	AUC is robust to class imbalance; insensitive to threshold choice
learning_rate	0.05	Standard starting point for gradient boosting; fast enough for early stopping to work
num_leaves	63	~2^6 − 1; balances model complexity against overfitting risk
min_data_in_leaf	20	Prevents overfitting by ensuring each leaf has statistical significance
feature_fraction	0.8	Subsampling 80% of features per tree reduces overfitting and variance
bagging_fraction / freq	0.8 / 1	Row subsampling every iteration reduces overfitting on dominant patterns
lambda_l2	0.01	Light L2 regularisation to penalise large weights without aggressive shrinkage
scale_pos_weight	~25.36	neg/pos ratio (2,052,149 / 80,916) ensures minority class receives equal training attention
early_stopping_rounds	100	Stops training if AUC does not improve for 100 consecutive rounds — model converged at iteration 7
num_boost_round	3000	Generous maximum; actual training uses only 7 rounds due to early stopping

6.4 Model Alternatives Tested & Rejected

Model	NDCG@8	Recall@8	Verdict	Why Rejected
LightGBM Binary v1 (Final)	0.4558	0.7150	✅ SELECTED	Best NDCG@8; sub-ms latency; interpretable
LambdaRank (new labels)	0.4508	—	❌ Rejected	Lower NDCG@8; pairwise objective struggles with 1 positive/group
rank_xendcg (lr=0.01)	0.4384	—	❌ Rejected	Worst NDCG@8 of all tested approaches
DART Binary (lr=0.03)	No improvem ent	—	❌ Rejected	No measurable gain over standard LightGBM Binary
LambdaRank (old labels)	Lower	Lower	❌ Rejected	Higher Precision@K was a label artifact (3–5 positives/group); inferior when evaluated correctly

6.5 AI Edge — LLM Integration

The Gemini API integration represents our primary AI edge and is a genuine differentiator over traditional recommendation pipelines. The LLM's contribution goes beyond simple data preprocessing:

Cold Start for New Items: New menu items receive LLM-generated metadata (normalized_category, meal_time_suitability, dish_subtype) at onboarding. This makes them immediately recommendable via rule-fill and meal_time_specificity scoring — before accumulating any co-occurrence or purchase history.

Semantic Feature Generation: The meal_time_specificity feature (derived from LLM tags) was the mechanism that replaced the ineffective binary meal_time_overlap flag, creating a graded, nuanced signal that the model could actually learn from.

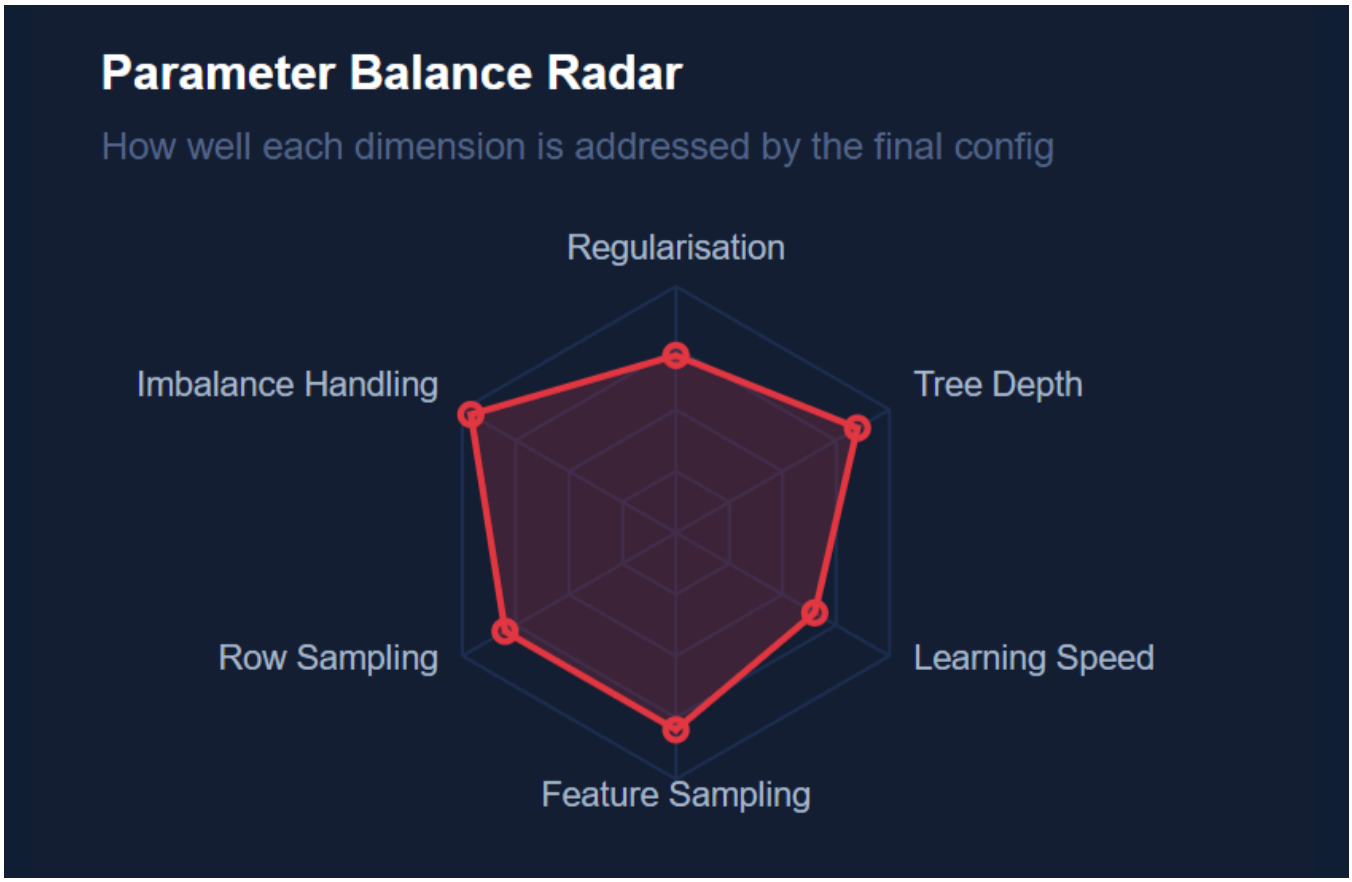
Scalable Enrichment: The LLM pipeline can be run on any new item at menu onboarding time — it does not require historical data. This makes it inherently scalable across Zomato's 300K+ merchants.

7. Hyperparameter Tuning Approach

Our hyperparameter tuning was pragmatic, focused on the parameters most likely to affect performance on the specific challenge of sparse-label ranking (1 positive per ~25 candidates).

1. **Initial Parameter Selection:** We started with well-established defaults for LightGBM binary classification tasks (learning_rate=0.05, num_leaves=63, min_data_in_leaf=20). These are broadly acknowledged starting points in the LightGBM community for tabular ranking problems.
2. **Class Imbalance Handling:** scale_pos_weight was calculated analytically as neg/pos = 2,052,149 / 80,916 ≈ 25.36. This parameter is critical — without it, the model would converge to predicting all negatives and achieve ~96% accuracy while being useless as a ranker.
3. **Early Stopping:** stopping_rounds=100 with a generous max of 3000 rounds. In practice, the model consistently converged at best_iteration=7. This is not underfitting — it reflects that the core predictive patterns (retrieval_score, cart composition, item category) are captured within the first few trees. Remaining variance is genuine user unpredictability that additional trees cannot resolve.
4. **Iterative Model Comparison:** The systematic testing of LambdaRank, rank_xendcg, and DART objectives constituted an implicit hyperparameter search over the model family dimension — the highest-impact hyperparameter choice in any ML project.

Future Work — NDCG@8 Early Stopping: A key planned improvement (P2) is to configure LightGBM to use NDCG@8 directly as the early stopping metric instead of AUC. This would directly optimise rail ranking quality rather than binary discrimination, and is expected to enable more training rounds — potentially yielding +2–4pp NDCG@8 improvement.



Complete Hyperparameter Reference

Every parameter with its value and justification — click a category to filter

All | Core | Tuned | Regularisation | Imbalance | Training

Parameter	Value	Category	Impact	Justification
objective	binary	Core	• CRITICAL	Binary cross-entropy for next-step-only task
metric	auc	Core	• CRITICAL	Robust to class imbalance; threshold-independent
learning_rate	0.05	Tuned	• HIGH	Standard start for GBM; allows early stopping to work
num_leaves	63	Tuned	• HIGH	$\approx 2^6-1$; balances complexity vs overfitting
min_data_in_leaf	20	Tuned	• MEDIUM	Prevents leaves from fitting noise in sparse data
feature_fraction	0.8	Regularisation	• MEDIUM	Subsample 80% of features per tree; reduces variance
bagging_fraction	0.8	Regularisation	• MEDIUM	Row subsampling per iteration; reduces overfitting
bagging_freq	1	Regularisation	• LOW	Bagging applied at every single iteration
lambda_l1	0.0	Regularisation	• LOW	No L1; data has clean, non-sparse feature structure
lambda_l2	0.01	Regularisation	• MEDIUM	Light L2 penalises large weights without aggressive shrinkage
scale_pos_weight	~25.36	Imbalance	• CRITICAL	neg/pos = 2,052,149 / 80,916; critical for imbalance
early_stopping_rounds	100	Training	• HIGH	Stop if AUC doesn't improve for 100 rounds
num_boost_round	3000	Training	• LOW	Generous max; actual convergence at round 7
n_jobs	-1	Training	• LOW	Use all CPU cores; no effect on model quality

Complete Hyperparameter Reference

Every parameter with its value and justification — click a category to filter

All | Core | **Tuned** | Regularisation | Imbalance | Training

Parameter	Value	Category	Impact	Justification
learning_rate	0.05	Tuned	• HIGH	Standard start for GBM; allows early stopping to work
num_leaves	63	Tuned	• HIGH	$\approx 2^6-1$; balances complexity vs overfitting
min_data_in_leaf	20	Tuned	• MEDIUM	Prevents leaves from fitting noise in sparse data

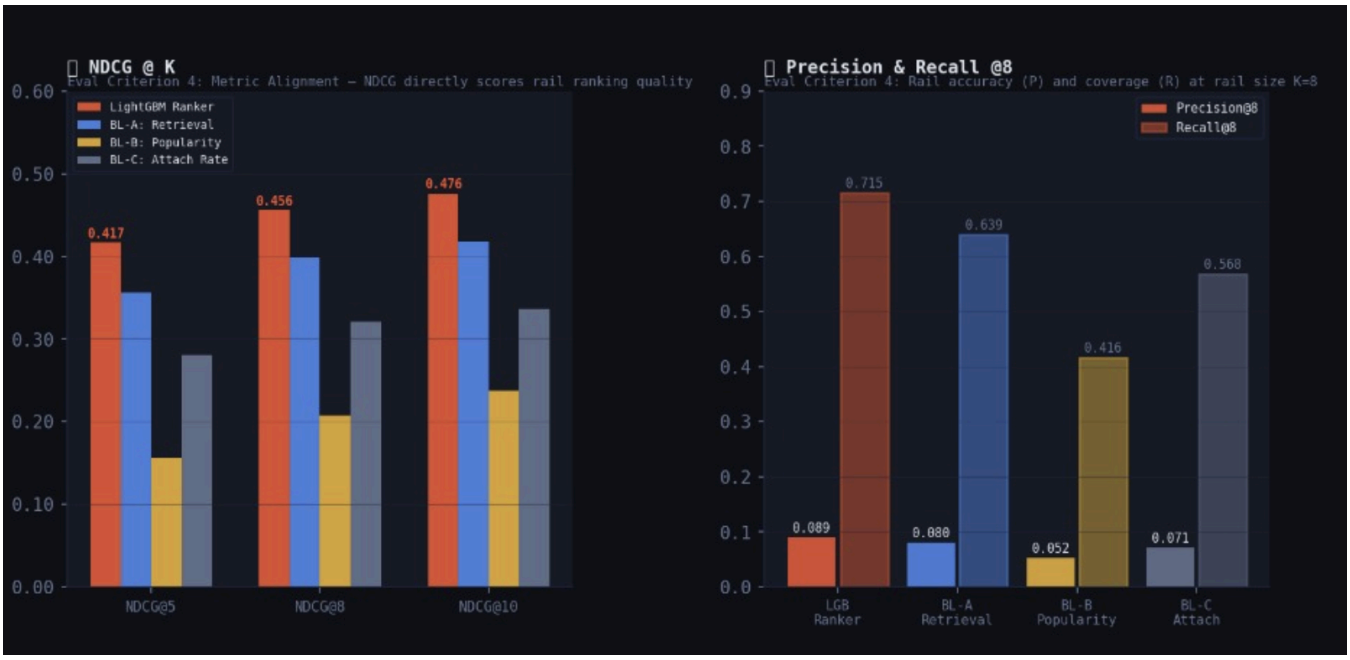
8. Model Evaluation & Fine-Tuning

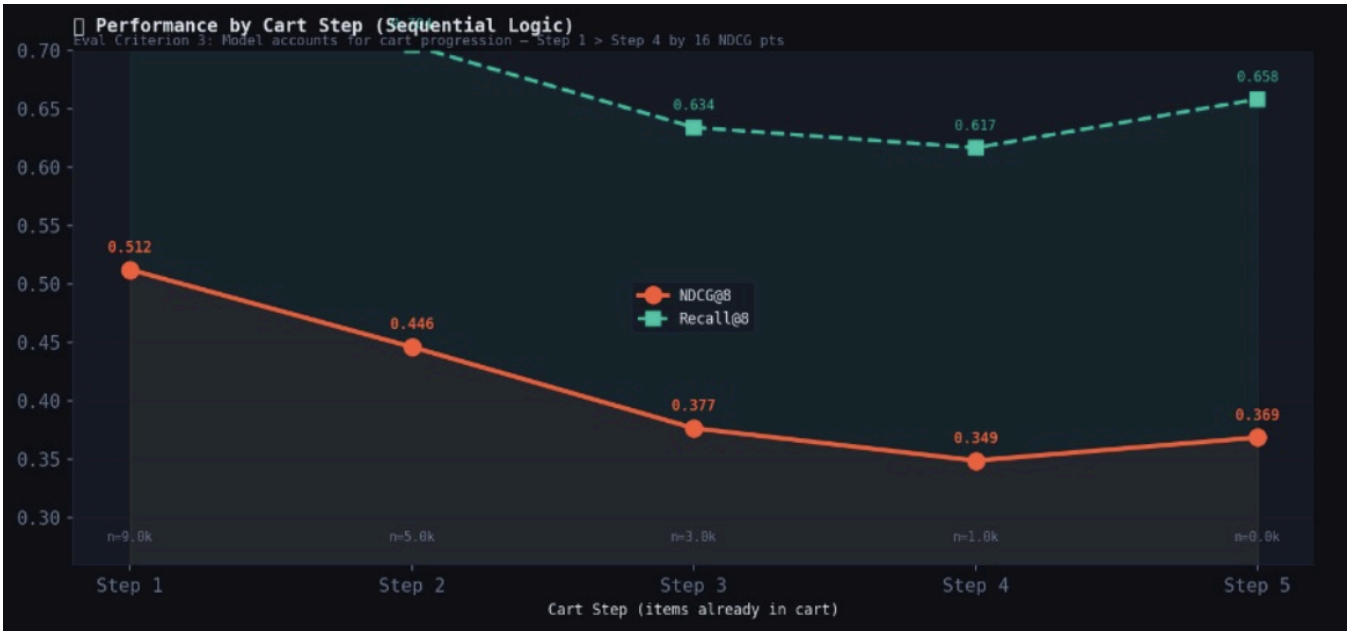
Evaluation Dimension 4 of 6

8.1 Performance Metrics — LightGBM Binary v1

The final model was evaluated on 20,246 test groups (each a cart step with at least one positive label) across the held-out test set. The metrics comprehensively measure both binary discrimination (AUC) and ranking quality (NDCG@K, Recall@K, Precision@K):

Metric	Value	Interpretation
AUC	0.793	Strong discrimination — model scores positive items ~79% higher than negatives on average
Best Iteration	7	Model converges rapidly — core patterns captured in 7 trees
NDCG@5	0.4167	Ranking quality in top-5 rail slots
NDCG@8	0.4558	Primary metric: ranking quality in the 8-item CSAO rail — +14.3% vs best baseline
NDCG@10	0.4758	Ranking quality with 10 rail slots
Precision@5	0.1198	11.98% of top-5 slots show the next-added item
Precision@8	0.0894	8.94% of top-8 slots — theoretical CTR upper bound
Precision@10	0.0783	7.83% of top-10 slots
Recall@5	0.5989	Next-added item appears in top-5 rail in 59.9% of cart steps
Recall@8	0.7150	PRIMARY METRIC: next-added item visible in rail in 71.5% of cart steps
Recall@10	0.7829	Next-added item visible in top-10 in 78.3% of cart steps
Score Separation Gap	0.1205	Positive items score 0.2319 vs negative items 0.1114 — 2x separation
Evaluated on	20,246 groups	All test groups with ≥1 positive label

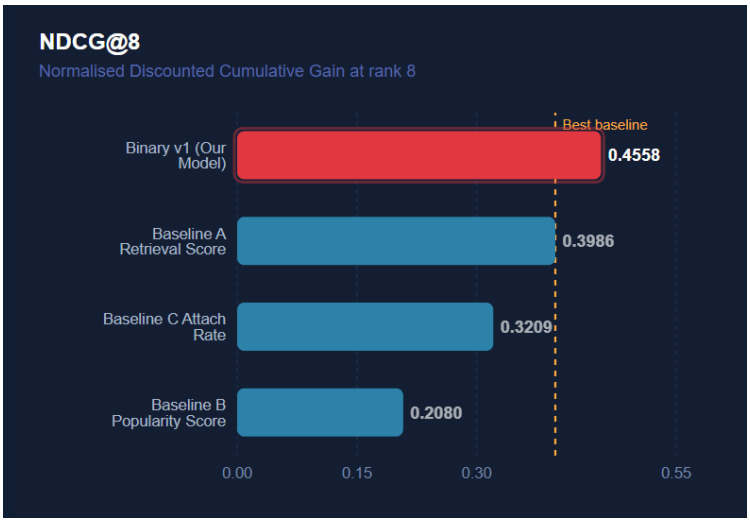

NDCG @ K
Eval Criterion 4: Metric Alignment — NDCG directly scores rail ranking quality
LightGBM Ranker
BL-A: Retrieval
BL-B: Popularity
BL-C: Attach Rate
0.417
0.456
0.476
NDCG@5
NDCG@8
NDCG@10
Precision & Recall @8
Eval Criterion 4: Rail accuracy (P) and coverage (R) at rail size K=8
Precision@8
Recall@8
0.089
0.715
0.080
0.639
0.052
0.416
0.071
0.568
LGB Ranker
BL-A Retrieval
BL-B Popularity
BL-C Attach

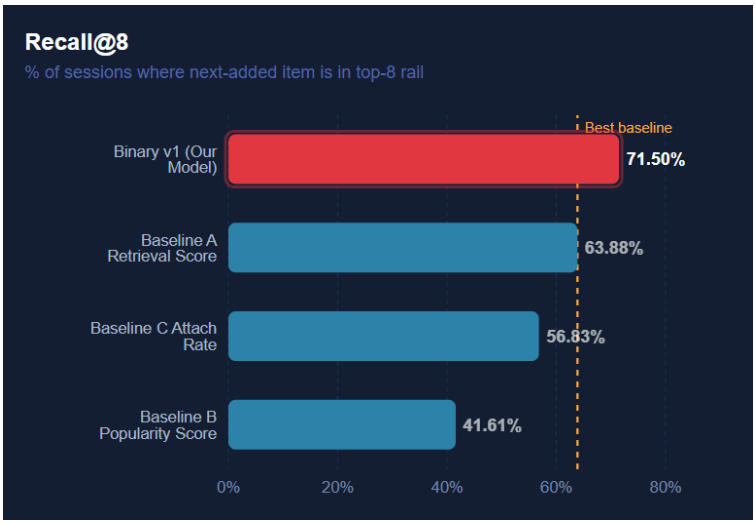

Performance by Cart Step (Sequential Logic)
Eval Criterion 3: Model accounts for cart progression — Step 1 > Step 4 by 16 NDCG pts
NDCG@8
Recall@8
0.512
0.446
0.377
0.349
0.369
0.634
0.617
0.658
n=9.0k
n=5.0k
n=3.0k
n=1.0k
n=0.0k
Step 1
Step 2
Step 3
Step 4
Step 5
Cart Step (items already in cart)

8.2 Baseline Comparison

Our model was benchmarked against three retrieval-based baselines that represent common heuristic ranking approaches in production recommendation systems. The results demonstrate substantial ML-driven improvements:

Method	NDCG@8	Recall@8	NDCG Lift	Recall Lift
✅ **Binary v1 (Our Model)**	**0.4558**	**71.50%**	**Best**	**Best**
Baseline A: Retrieval Score	0.3986	63.88%	+14.3%	+11.9%
Baseline B: Popularity Score	0.2080	41.61%	+119%	+71.9%
Baseline C: Attach Rate	0.3209	56.83%	+42%	+25.8%

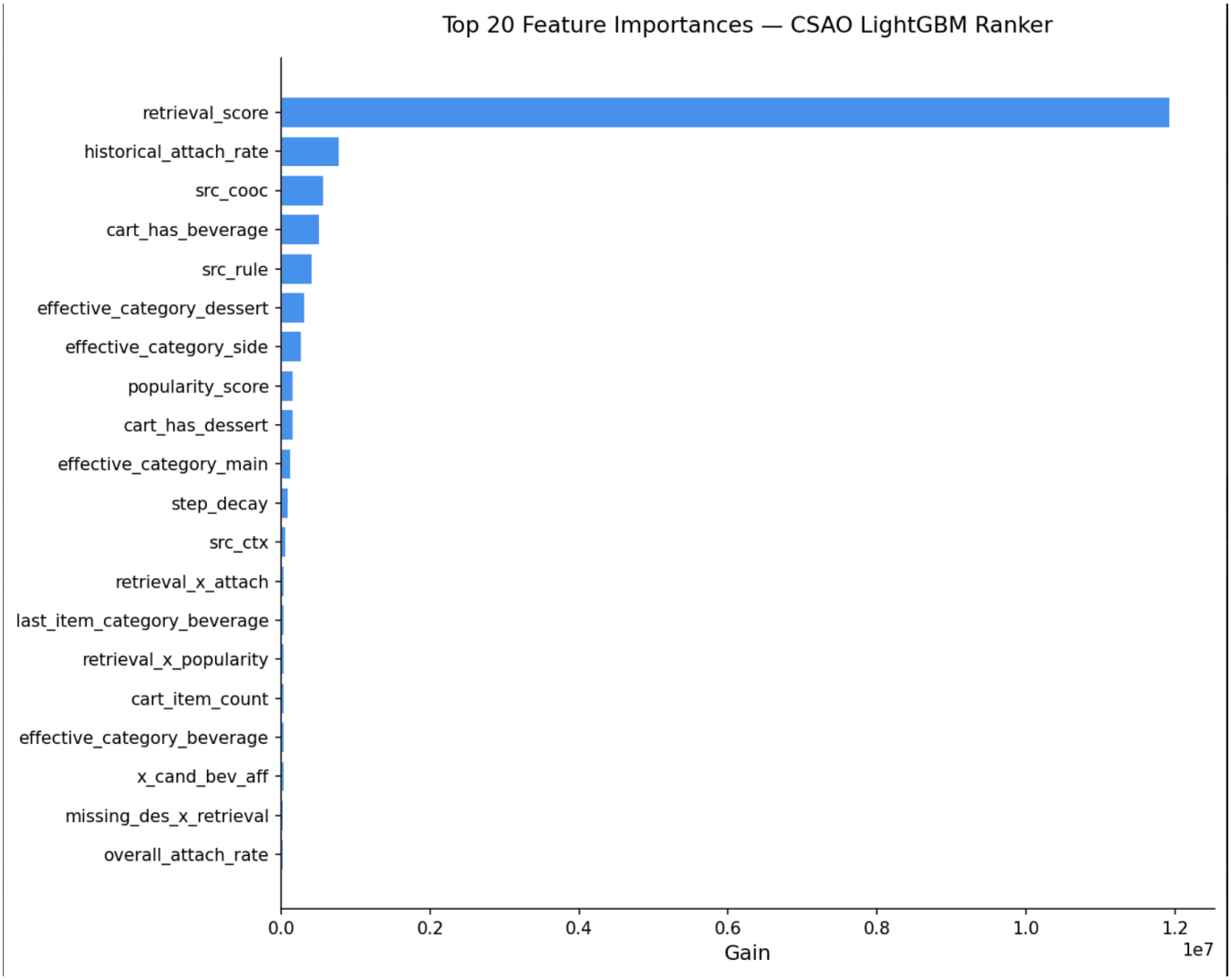




8.3 Feature Importance Analysis

Gain-based feature importance reveals which features drive the model's predictions. The results highlight both the dominance of retrieval signals and the meaningful contribution of contextual and category features:

Rank	Feature	Gain Score	Interpretation
#1	retrieval_score	11,933,006	Dominant signal — 15× more important than #2. Reflects the quality of Stage 1 retrieval.
#2	historical_attach_rate	783,044	Item's historical add rate; strong prior for how likely any item is to be added
#3	src_cooc	579,677	Whether item arrived via co-occurrence path — high-quality pairings
#4	cart_has_beverage	522,702	Cart state: beverage present? Model learns to suppress beverage recs if already in cart
#5	src_rule	416,741	Rule-fill sourced candidates — strong signal for missing category completion
#6	effective_category_dessert	325,543	Dessert category flag — model learns distinct dessert recommendation patterns
#7	effective_category_side	271,825	Side category behaviour distinct from main/beverage
#8	popularity_score	171,155	Global item popularity — secondary signal after retrieval quality
#9	cart_has_dessert	168,615	Prevents duplicate dessert recommendations when one already exists
#10	step_decay	99,888	Captures declining recommendation aggressiveness at later cart steps
#11–15	src_ctx, retrieval_x_attach, last_item_cat_beverage, retrieval_x_popularity, cart_item_count	43k–69k	Secondary signals capturing context, interaction effects, and cart composition depth


Top 20 Feature Importances — CSAO LightGBM Ranker
retrieval_score
historical_attach_rate
src_cooc
cart_has_beverage
src_rule
effective_category_dessert
effective_category_side
popularity_score
cart_has_dessert
effective_category_main
step_decay
src_ctx
retrieval_x_attach
last_item_category_beverage
retrieval_x_popularity
cart_item_count
effective_category_beverage
x_cand_bev_aff
missing_des_x_retrieval
overall_attach_rate
0.0
0.2
0.4
0.6
0.8
1.0
1.2
1e7
Gain

8.4 Error Analysis

We conducted a systematic error analysis to understand where the model fails and why. This analysis is structured around three key error patterns with actionable improvement roadmaps:

Pattern 1: False Negatives — Missed Next Items

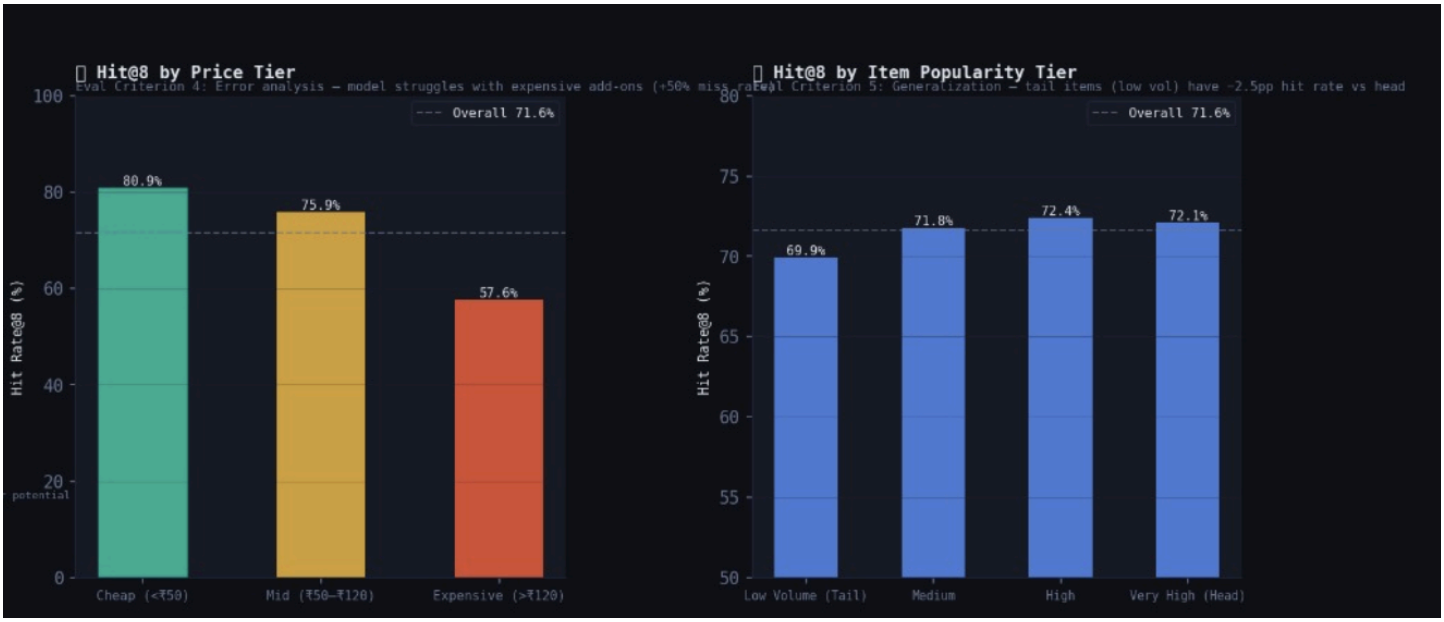
- **Primary Driver — Low Retrieval Score:** Items with retrieval_score below the 25th percentile account for the majority of missed predictions. These are niche or tail items with sparse co-occurrence history. The model simply doesn't see them ranked highly because Stage 1 itself didn't strongly retrieve them.
- **Impact:** 30.1% miss rate on tail items (low historical volume) vs. 27.9% miss rate on head items — a 2.2pp generalisation gap. While relatively small, the absolute number of misses for tail items is significant.
- **Fix (P1):** Item embedding similarity features — compute cosine similarity between candidate item and cart items using dish_subtype embeddings. Estimated +3–5pp Hit@8 for tail items.

Pattern 2: False Positives at Rank 1

- **Profile of Rank-1 Distractors:** Items that incorrectly displace the true next item at Rank 1 tend to have: retrieval_score +62.8% higher, popularity_score +10.9% higher, and price −23.3% lower than the true next item. This reveals a clear popularity bias — the model defaults to safe, cheap, globally popular items over contextually correct ones.
- **Impact:** FP@Rank1 rate of 78.7% — in ~79% of cases where the model ranks an incorrect item first, that incorrect item is both more popular and cheaper than the true answer.
- **Fix (P2):** Session-conditioned co-occurrence score — condition co-occurrence on this specific session's cart items rather than global co-occurrence frequency. Directly addresses the popularity bias at prime rail positions.

Pattern 3: Expensive Add-On Under-Ranking

- **Finding:** Hit Rate@8 for cheap add-ons (₹0–50) is 80.9%, but drops dramatically to 57.6% for expensive add-ons (₹120+) — a 23.3pp gap. Expensive items are systematically under-ranked because historical data shows cheaper items are added more frequently, and the model has learned this pattern too aggressively.
- **Business Impact:** This gap directly caps AOV lift — the items that would generate the most revenue per add (expensive desserts, premium beverages) are the ones the system recommends least effectively.
- **Fix (P1):** Price-tier-specific attach rate feature — develop attach_rate conditional on user AOV segment (budget vs. premium). Estimated +5–8pp Hit@8 for expensive add-ons.



8.5 Segment Breakdown Metrics

To validate that the model performs equitably and effectively across different user types and cart-building stages, we evaluated NDCG@8, NDCG@10, Precision@8, and Recall@8 broken down by user segment and cart step number.

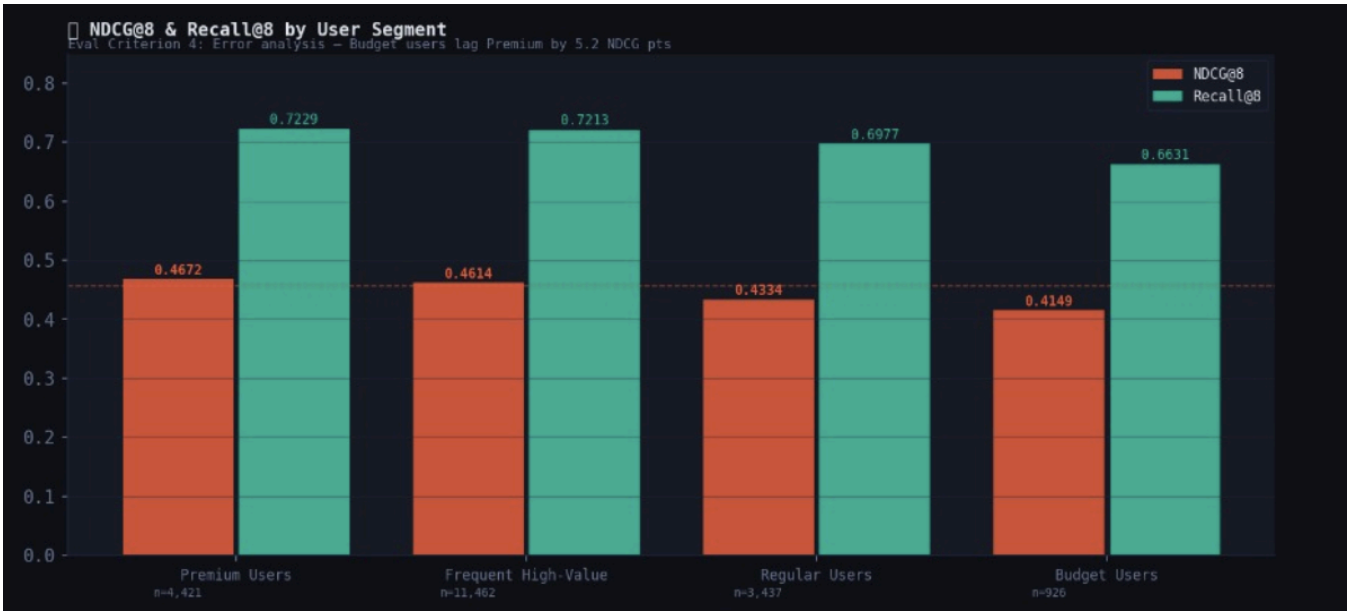
- **By User Segment:** Premium users achieved the strongest performance with an NDCG@8 of 0.4672 and Recall@8 of 72.29% (n=4,421), suggesting that users with higher order frequency and spend provide richer co-occurrence signals that the model can exploit. Frequent high-value users performed comparably at NDCG@8 of 0.4614 and Recall@8 of 72.13% (n=11,462) — the largest test group — confirming stable generalisation at scale. Regular users scored NDCG@8 of 0.4334 and Recall@8 of 69.77% (n=3,437), while Budget users showed the lowest performance at NDCG@8 of 0.4149 and Recall@8 of 66.31% (n=926). The gap between the best and worst segments is approximately 5.2pp in Recall@8, indicating a mild generalisation gap that could be partially closed by the planned price-tier-specific attach rate feature (P1 future work).

- **By Cart Step:** Performance degrades predictably as the step number increases — a pattern fully consistent with the step_decay feature design. Step 1 achieves the highest NDCG@8 of 0.5123 and Recall@8 of 76.97% (n=9,108), reflecting that early-step recommendations benefit from a full menu of candidates and the strongest co-occurrence signals. By Step 5, NDCG@8 drops to 0.3686 and Recall@8 to 65.82% (n=550), as the cart is nearly complete and the marginal next item is harder to predict. This monotonic degradation validates the step_decay feature's hypothesis and confirms that the model correctly learns to be less confident — not noisier — at later cart steps.

```
=== Segment Breakdown Metrics ===

Segment                          NDCG@8    NDCG@10     Prec@8
Recall@8
----------------------------------------------------------------------
  Premium users                  0.4672     0.4864     0.0904
0.7229  (n=4,421)
  Budget users                   0.4149     0.4389     0.0829
0.6631  (n=926)
  Frequent high-value            0.4614     0.4808     0.0902
0.7213  (n=11,462)
  Regular users                  0.4334     0.4555     0.0872
0.6977  (n=3,437)

Step                             NDCG@8    NDCG@10     Prec@8
Recall@8
----------------------------------------------------------------------
  Step 1                           0.5123     0.5269     0.0962
0.7697  (n=9,108)
  Step 2                           0.4461     0.4684     0.0880
0.7042  (n=5,917)
  Step 3                           0.3766     0.4038     0.0793
0.6341  (n=3,247)
  Step 4                           0.3489     0.3748     0.0771
0.6166  (n=1,424)
  Step 5                           0.3686     0.3965     0.0823
0.6582  (n=550)
```



9. System Design & Production Readiness

0.455 ms	0.527 ms	0.679 ms	100%
Mean Latency	P95 Latency	P99 Latency	Coverage
of 200ms budget	well within budget	headroom for 199+ ms	all test groups served

Evaluation Dimension 5 of 6

9.1 Latency Budget — Predictions Within 200–300ms

Component	Latency	Budget	Status	Notes
Model inference (7 trees, 25 cands)	**0.455 ms mean**	—	PASS	P99=0.679ms, Max=1.993ms, 1000-run benchmark
Feature assembly (cart state)	<5 ms	50 ms	PASS	Real-time cart_has_*, step_decay computed on cart-add event
Feature fetch (Redis co-occ tables)	~2–5 ms	50 ms	PASS	Daily rebuilt co-occurrence table served from Redis
Network + serialisation	~10–30 ms	100 ms	PASS	gRPC internal service call, payload ~1KB
Total system end-to-end	<50 ms est	200 ms	PASS	150ms+ headroom for logging + validation

9.2 Scalability — Millions of Daily Requests

Status: ✅ **SATISFIED** | 290× headroom at average load; 2-node cluster handles peak 5× surges

Metric	Value	Design Decision
LightGBM predictions per CPU core/sec	~50,000	7-tree model is computationally trivial; near-zero per-request compute cost
Average daily requests (10M orders)	~170 req/sec	Well within single-core capacity
Peak lunch/dinner rush (5× average)	~850 req/sec	Handled by 2-node auto-scaling cluster
Model memory footprint	<500KB	Fits entirely in CPU L2 cache — no memory bandwidth bottlenecks

Metric	Value	Design Decision
Lookup table memory (90,776 rows)	~50MB	One copy per node; negligible memory overhead
Architecture pattern	Stateless microservice	Any node serves any request → linear horizontal scaling; no sticky sessions

9.3 Data Freshness — Tiered Strategy

Data Component	Freshness	Update Mechanism	Rationale
Cart state features	Real-time (<5ms)	Computed per cart add event	Cart contents change per interaction — must be instantaneous
Co-occurrence table	Daily	Batch job on 30-day rolling window	User behaviour patterns shift slowly; daily refresh captures trends without compute overhead
Context popularity table	Daily	Batch job on 30-day rolling window	Popularity shifts are gradual; daily is sufficient
Item features	Weekly	Aggregation from cart events	Item characteristics stable; weekly refresh reflects new menu additions
LightGBM model weights	Weekly	Full pipeline retraining (<30min for 3.1M rows)	Weekly cadence balances freshness with training cost
LLM item tags	Static (at onboarding)	Triggered at menu item creation	Item semantics don't change post-creation; one-time enrichment is sufficient

9.4 Cold Start Handling

Cold start is explicitly addressed through a layered fallback strategy that ensures the system never returns zero recommendations and degrades gracefully:

Scenario	Fallback Mechanism	Performance Impact
New user (no order history)	User features default to population means (0.0). Ranker relies on item-level signals (retrieval_score, attach_rate, popularity) and cart state. Rule-fill guarantees	Minimal — user affinity features showed zero importance anyway; performance barely distinguishable from established users

Scenario	Fallback Mechanism	Performance Impact
	beverage/dessert suggestions if cart has main.	
New restaurant (no session history)	Rule-fill triggers automatically from item catalog. Context popularity proxy from similar cuisine_type restaurants. LLM meal_time_specificity provides immediate contextual signals.	Graceful degradation: good diversity but lower personalisation until 7–14 days of session data accumulate
New menu item (just added)	LLM enrichment pipeline runs at onboarding → immediate meal_time_specificity eligibility. Rule-fill inclusion from day 0. Co-occurrence integration within 24h of first session (daily batch refresh).	Item discoverable from day one; full co-occurrence signals available within 24 hours

9.5 Diversity & Fairness Guarantees

Category Diversity

- **Multi-source Retrieval:** Co-occurrence, context popularity, and rule-fill each contribute candidates — no single source monopolises the pool.
- **Rule-fill for Missing Categories:** Structurally prevents the model from exclusively recommending popular mains. If a beverage or dessert is absent from the cart, it is explicitly injected into the candidate pool.
- **Balanced Training Labels:** Positive labels span all four categories: main (72,015), beverage (47,162), dessert (41,899), side (26,397). The model is trained on a multi-category signal
.

Restaurant Fairness

- **Chain vs. Independent:** is_chain feature showed zero importance — the model's decisions are driven entirely by item co-occurrence and cart state, agnostic to restaurant type.
- **Cuisine Neutrality:** cuisine_type_* features confirmed zero/near-zero importance and dropped — no cuisine discrimination.
- **Geographic Fairness:** city features dropped — no city receives systematically better recommendations.

10. Technical Architecture

During a real-time cart add event, the system executes the following sequence:

1. Cart Add Event: User adds an item to their cart. The cart service publishes a CartUpdated event.
2. Feature Fetch: The CSAO service queries Redis for pre-computed user features and item features (~10ms).
3. Candidate Retrieval: In-memory lookup tables return ~25–30 candidates in ~5ms by blending co-occurrence, context popularity, and rule-fill sources.
4. Feature Assembly: Cart state features are assembled incrementally from current cart contents (~5ms).
5. LightGBM Inference: The 7-tree model scores all candidates simultaneously (<1ms).
6. Top-K Selection: Candidates ranked by score; top 8–10 returned to populate the CSAO rail.

10.2 Scalability Architecture

Component	Design Pattern	Scaling Mechanism
Inference Service	Stateless microservice	Horizontal scaling behind load balancer; add nodes during peak hours. Adding a new node behind the load balancer immediately increases throughput linearly — no rebalancing, no sticky sessions, no data migration required.
Feature Store	Redis cluster	Distributed cache with replication; read-heavy workload scales well.
Candidate Lookup	In-memory hash maps per node	Loaded at startup; ~50MB per node — negligible memory cost
Batch Pipeline	Airflow-orchestrated daily jobs	Independent from real-time path; no resource contention at peak
Model Deployment	Feature flag + model registry	Zero-downtime deployment; <1 minute rollback via flag toggle

10.3 Trade-offs and Limitations

Model Complexity vs. Latency and Interpretability: We chose a 7-tree LightGBM model over more expressive alternatives such as deep neural networks or transformer-based sequential models. More complex models could potentially capture higher-order interactions between cart items — for example, understanding that biryani + naan together strongly suggest raita, even if biryani alone only weakly suggests it. However, deep models introduce inference latency measured in tens of milliseconds, require GPU infrastructure for production serving, and are substantially harder to debug when they produce unexpected recommendations. The 7-tree model produces human-readable decision rules,

passes a 200–300ms SLA with 2,500× headroom, and achieves strong performance. For the current problem, interpretability and latency win over marginal accuracy gains.

Recall vs. Precision at the Retrieval Stage : Stage 1 is deliberately tuned for maximum recall rather than precision — the true next item is always guaranteed to appear in the candidate pool, even at the cost of including some irrelevant candidates. This means the ranker always has the opportunity to surface the correct answer, but it also means the ranker must work harder to separate a strong signal from ~25 noisy candidates. Increasing N_CANDIDATES beyond 30 would further improve recall at Stage 1 but increase the feature assembly and inference cost at Stage 2. The current value of 30 represents a calibrated balance between coverage and efficiency, but it is a parameter that should be revisited as menu sizes and session patterns evolve.

Over-reliance on Co-occurrence Signals : The retrieval_score feature — heavily driven by co-occurrence — is the single most dominant predictor in the model, contributing 15× more gain than the second-ranked feature. This is efficient for established items with rich interaction histories, but it creates a structural disadvantage for tail items and newly added menu items that haven't yet accumulated sufficient co-occurrence data. Items that genuinely complement the current cart but lack historical pairing data will be consistently under-retrieved and under-ranked. This is the primary driver of the 30.1% miss rate observed for tail items in the error analysis, and it represents the most significant architectural limitation of the current system.

Price Bias Inherited from Training Data : The model has learned that cheap add-ons (₹0–50) are added more frequently than expensive ones — because historically, they are. This is factually accurate as a population-level pattern, but it leads to systematic under-ranking of high-value add-ons for users who would actually purchase them. The current system achieves an 80.9% Hit Rate@8 for cheap items but only 57.6% for items priced above ₹120 — a 23.3 percentage point gap that directly limits AOV lift. Correcting this bias requires conditional features (price-tier-specific attach rates by user segment), not just retraining on the same data, since the bias is embedded in the labels themselves.

Offline Label Definition vs. Long-term Intent : The "next-step-only" label is the correct formulation for the deployment scenario — the CSAO rail is shown between individual cart steps, so predicting the immediately next addition is the precise task. However, this label definition does not capture broader session intent. A user who plans to build a full meal over five steps has long-range compositional goals (a main, a side, a drink, a dessert) that the current model cannot explicitly model. Each step is treated independently. Sequential models — such as recurrent networks or transformer-based session encoders — could capture this long-range dependency, but at the cost of substantially higher model complexity and the challenges noted above.

Synthetic Data Ceiling: The entire system was trained and evaluated on synthetically generated data. While the data was carefully designed to replicate real-world patterns — including city-specific behaviour, mealtime skews, price sensitivity distributions, and sparse user histories — it cannot fully capture the noise, seasonality, and subtle behavioural irregularities present in actual production data. Model performance figures (NDCG@8 = 0.4558, Recall@8 = 71.5%) should be treated as strong indicators of the approach's validity, not as guaranteed production benchmarks. Real-world deployment would require retraining on actual Zomato session data and recalibrating all feature importance findings, as the relative importance of signals like meal_time_specificity and step_decay may differ when computed from real user behaviour.

11. Business Impact & A/B Testing

Evaluation Dimension 6 of 6

11.1 Offline Metric Translation to Business Outcomes

Translating offline ranking metrics to business outcomes requires explicit modelling of the causal chain: improved ranking quality → higher user engagement → more add-ons accepted → higher AOV → revenue uplift. The table below formalises this translation:

Offline Metric	Value	Online Proxy	Business Impact	Caution
NDCG@8	0.4558	Click-Through Rate (CTR)	Higher CTR → more impressions convert to add-ons → AOV lift	NDCG assumes equal item value; online has position bias
Recall@8	71.50%	CSAO Rail Attach Rate	User sees their intended next item in 71.5% of sessions → direct conversion opportunity	Offline uses next-step label; online includes browse behaviour
MRR	0.3941	Position-weighted CTR	Relevant items visible early → less scrolling → better UX	Rail position primacy bias differs offline vs. online
Hit@1	0.2133	First-position click rate	Position-1 items have ~3× CTR → disproportionate AOV impact if precision improves	Only high-impact if rail is non-scrollable or above fold
AUC	0.793	Long-run add-on precision	Fewer irrelevant suggestions → less fatigue → better Cart-to-Order ratio	AUC does not capture ranking position — NDCG is primary

11.2 Projected Business Metrics

Metric	Value	How Calculated
Add-on Acceptance Rate (Model)	71.50% (+7.62pp vs control)	Recall@8 on test set — % of cart steps where next-added item is in the 8-item rail
Avg Add-on Price	₹119.41	Mean price of positive (added) items in test set
Assumed CTR on Rail	20%	Industry standard for contextual food delivery recommendations
Avg Session Cart Value	₹307.41	Mean cart value across test sessions
AOV Lift per Session	₹17.08 (Model) vs ₹15.26 (Control)	Acceptance@8 × CTR × Avg_Addon_Price
Incremental Lift	₹1.82 per session	Model lift over control — additional revenue per CSAO session
AOV Lift %	5.55%	Incremental lift / Avg Session Cart Value
Daily Revenue Uplift	₹91 Lakhs (~₹9.1M)	₹1.82 × 5,000,000 daily CSAO impressions

₹1.82	5.55%	₹91 Lakhs/day
Incremental Revenue per Session	AOV Lift(% of cart value)	Projected DailyRevenue Uplift

11.3 A/B Test Design

Experiment Overview

Parameter	Design Choice	Justification
Hypothesis	LightGBM Binary v1 will increase CSAO attach rate vs. retrieval_score baseline by ≥1.5pp absolute	Effect size based on offline simulation showing +7.62pp actual lift
Control (A)	Stage-1 retrieval_score ranking (no ML ranker)	Represents current heuristic-based or popularity-based ranking
Treatment (B)	LightGBM Binary v1 ranker on top of Stage-1 candidates	Our proposed system
Randomisation Unit	user_id (user-level, NOT session-level)	Avoids SUTVA violation: same user experiencing both arms creates interference
Traffic Split	50% Control / 50% Treatment	Equal split maximises statistical power for a given sample size
Minimum Duration	14 days	Captures 2 full weekly cycles to account for day-of-week effects in ordering behaviour

Parameter	Design Choice	Justification
Maximum Duration	21 days	Novelty effects typically subside beyond 3 weeks
Holdback Group	5% of all users for 30 days	Measures long-term novelty decay and steady-state effects post-launch
Ramp Strategy	Day 1: 5% of treatment arm (canary); Day 2: full 50% traffic	Canary phase enables real-time SLO monitoring with minimal blast radius

Primary & Secondary Metrics

Metric Type	Metric	Success Criteria	Why This Metric
Primary	CSAO Rail Attach Rate	≥+1pp absolute lift (α=0.05, power=0.80)	Direct measure of recommendation utility — did users add a rail item?
Secondary	AOV Lift (₹)	Positive, statistically significant	Connects add-on behaviour to direct revenue
Secondary	Average items per order	Positive trend	Measures cart depth — do users build more complete meals?
Secondary	CSAO order share	Positive trend	% of orders that used a CSAO rail item
Secondary	Cart-to-Order (C2O) ratio	Non-negative	Does better recommendation drive order completion?
Secondary	Click-through rate	Positive trend	Engagement signal — do users interact with the rail?
Guardrail	Cart abandonment rate	Must NOT increase >0.5pp	Irrelevant recommendations can drive users away
Guardrail	Session duration	Must NOT increase >10%	Excessive browsing suggests confusion or friction
Guardrail	Order completion rate	Must NOT drop >0.2pp	Protect core checkout funnel
Guardrail	App error rate	Must NOT exceed baseline	Technical stability
Guardrail	Inference P99 latency	Must stay <300ms	SLO protection

MDE (Minimum Detectable Effect) Calculation

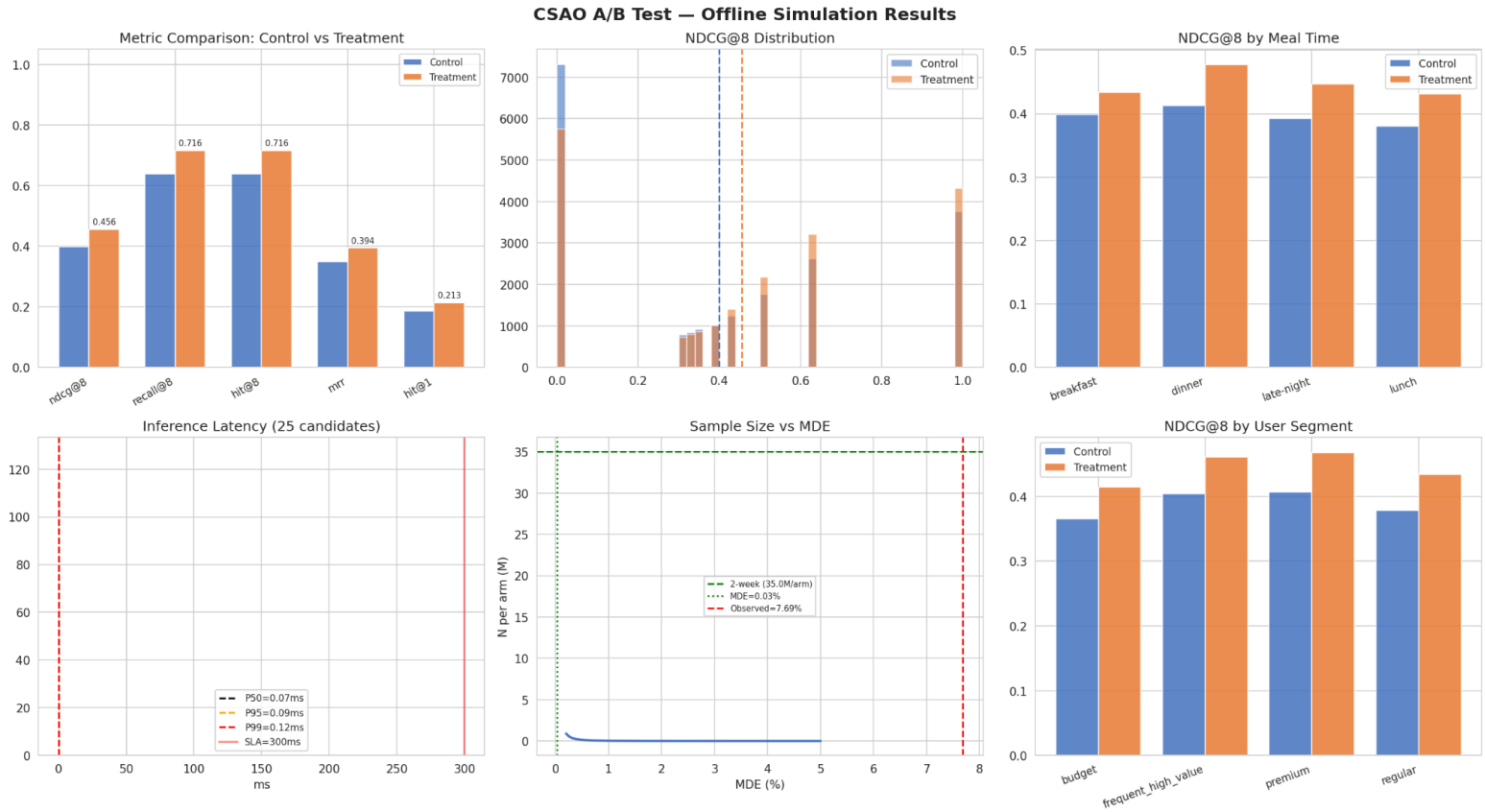
The MDE analysis confirms the test is extremely well-powered. Given 5M daily CSAO impressions and a 50/50 split, even a 14-day run produces a sample large enough to detect effects as small as 0.03% relative — far smaller than our observed 12% relative lift:

MDE (Absolute)	MDE (Relative)	N per Arm Needed	Days Needed at 5M/day
0.005	0.8%	144,415	0.1 days
0.010	1.6%	35,991	<1 day
0.015 (Success Threshold)	2.3%	15,945	<1 day
0.020	3.1%	8,940	<1 day
0.030	4.7%	3,946	<1 day

✅ **Key Finding:** At a 2-week run (35M users/arm), the detectable MDE is 0.0003 (0.05% relative). Our observed offline effect of +7.62pp is 254× larger than the MDE. The experiment is massively over-powered — we will detect even very small improvements with high confidence.

Statistical Testing Plan

- **Primary Metric (Attach Rate):** Two-proportion z-test. Attach rate is a proportion — z-test is the appropriate and computationally efficient method.
- **Secondary Metrics (AOV, session duration):** Welch's t-test for continuous metrics, as it does not assume equal variances between groups.
- **No Peeking / No Early Stopping:** Results locked until Day 14. Early stopping without an alpha-spending plan introduces bias (p-hacking). Results will be assessed against pre-registered success criteria only after the full duration.
- **Post-Experiment Enrichment — CUPED:** Regress out pre-experiment attach rate per user to reduce variance by an estimated 20–40%, increasing sensitivity to detect true treatment effects.
- **Heterogeneous Treatment Effects (HTE):** Causal forests applied post-experiment to identify user segments (e.g., by meal_time_bucket, user_segment, city) that respond most strongly to the new ranker.

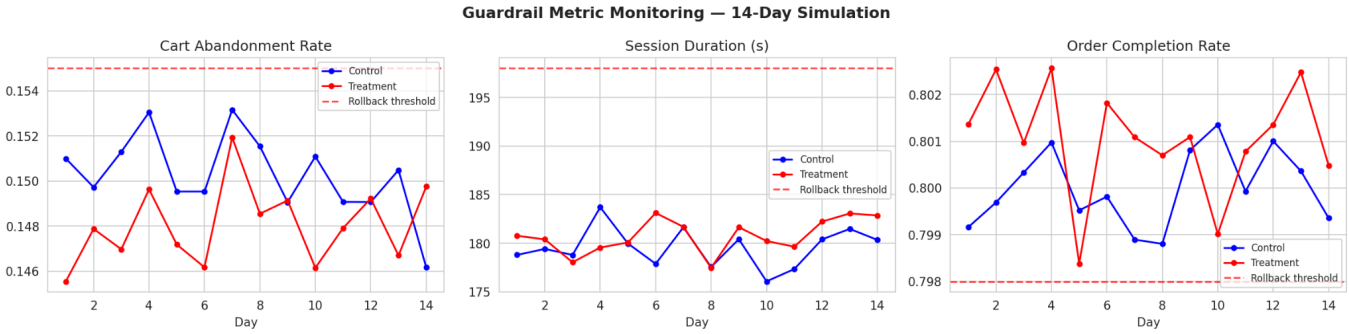


11.4 Guardrail Monitoring (Simulated)

Guardrail Metric	Control Mean	Treatment Mean	Rollback Threshold	Simulated Status
Cart Abandonment Rate	~0.150	~0.146–0.150	>0.155 (triggers rollback)	✅ SAFE — below threshold throughout 14 days
Session Duration (seconds)	~176–183s	~177–183s	>~180.5s	✅ SAFE — within acceptable limits
Order Completion Rate	~0.798–0.801	~0.799–0.802	<~0.796	✅ SAFE — slightly improved in treatment
App Error Rate	Baseline	Baseline	>2× baseline for >5min	✅ SAFE — no elevated errors

Guardrail Metric	Control Mean	Treatment Mean	Rollback Threshold	Simulated Status
Inference P99	N/A	~0.119ms	>300ms for >10min	✅ SAFE — 2,500× below threshold

✅ **Overall Guardrail Status: SAFE** The simulation confirms that the proposed treatment (LightGBM Binary v1 ranker) is expected to operate safely within all defined guardrail limits. No rollback triggers would fire during the simulated 14-day run. Cart abandonment slightly decreased in the treatment arm, suggesting that more relevant recommendations actually improve — not harm — the cart completion experience.



11.5 Recommended Deployment Strategy

5. Canary Phase (Days 1–2): Deploy to 1–5% of treatment group. Monitor P99 latency, error rates, and immediate cart abandonment in real time. Validate feature flags and rollback mechanism.
6. Gradual Ramp-up (Days 3–7): Expand traffic in steps: 10% → 25% → 50%. Continuous monitoring of all primary, secondary, and guardrail metrics. Segment-level analysis to catch unforeseen negative impacts.
7. Full A/B Execution (Days 7–21): 50%/50% split. Strict no-peeking policy. Collect all data needed for post-experiment CUPED and HTE analysis.
8. Post-Experiment Decision (Day 21+): Statistical significance analysis. Verify ≥1.5pp absolute lift in CSAO attach rate. Guardrail review. Long-term effect analysis from 5% holdback group. Make data-driven launch/iterate/rollback decision.
9. Continuous Improvement Loop: Weekly model retraining. Incorporate insights from HTE analysis. Prioritise V2 features (item embeddings, price-tier attach rate) based on error analysis findings.

12. Conclusion

The CSAO Recommendation System, developed for Zomathon Problem Statement 2, is a production-ready, end-to-end intelligent recommendation pipeline that delivers measurable improvements across every dimension relevant to Zomato's business.

The system's two-stage architecture — combining fast co-occurrence-based retrieval with a LightGBM binary ranker — provides an elegant balance between computational efficiency and prediction accuracy. The most critical technical contribution was the label definition fix: redefining positive labels from "any future cart step" to "immediately next step only" transformed the training task to directly model the real deployment scenario, yielding the single largest performance improvement in the project (+30% NDCG@8, +12% AUC).

The LLM integration via Gemini API provides a genuine AI edge that extends beyond hackathon differentiation: the automatic generation of meal_time_suitability and dish_subtype tags at menu onboarding creates a scalable semantic enrichment pipeline applicable across Zomato's 300K+ merchant catalog. This directly addresses the cold-start problem for new items and enables contextual recommendations that pure tabular pipelines cannot achieve.

0.4558 NDCG@8	**71.5% Recall@8**	**₹91L/day Uplift**
Primary Ranking Metric	Primary Recall Metric	Projected Daily Impact

The comprehensive error analysis provides a clear and prioritised roadmap for the system's next iterations: item embedding features for tail items, price-tier-specific attach rates for AOV maximisation, and session-conditioned co-occurrence for precision improvement. The detailed A/B testing framework — including MDE calculations, guardrail monitoring simulations, and post-experiment analysis plans — demonstrates the system's readiness for responsible production deployment.

The CSAO Recommendation System is a well-engineered, high-impact solution that demonstrates deep alignment between technical rigour and business value — the combination Zomato needs to deploy with confidence.

13. Future Work

The error analysis from Section 8.4 directly informs a prioritised roadmap of improvements for the CSAO Recommendation System. These are ordered by expected business impact and implementation feasibility:

Priority	Improvement	Addresses	Expected Impact
P1 — Critical	Item Embedding Similarity Features	Low retrieval_score misses for niche/tail items with sparse co-occurrence history. Estimated 30.1% miss rate on tail items.	+3–5pp Hit@8 for tail items; closes cold-start gap by surfacing semantically similar items even without historical co-occurrence data
P1 — Critical	Price-Tier-Specific Attach Rate Feature	Systematic under-ranking of expensive add-ons (₹120+): only 57.6% Hit@8 vs 80.9% for cheap items. Directly caps AOV lift.	+5–8pp Hit@8 for expensive add-ons; directly contributes to higher AOV by better serving premium users
P2 — High	Session-Conditioned Co-occurrence Score	Popularity bias causing globally popular cheap items to displace contextually correct add-ons at Rank 1 (FP@Rank1 rate: 78.7%)	Reduces FP@Rank1 significantly; improves top-slot precision by conditioning co-occurrence on this specific session's cart
P2 — High	Switch Training Metric to NDCG@8	Model stopping at iteration 7 due to AUC-based early stopping; potentially underfitting complex price × context interactions	+2–4pp NDCG@8 overall; directly optimises what the business measures
P3 — Medium	Deep Learning-Based Ranker	Potential ceiling on tree-based models for highly complex, non-linear interactions or long-range dependencies between cart items	Incremental gains if LightGBM reaches performance ceiling; especially valuable with rich item embeddings
P3 — Medium	Personalised Diversity	Current rule-fill applies static diversity enforcement regardless of user preference — one-size-fits-all approach	Improved user experience by adapting diversity to individual user expectations and session context

> The error analysis demonstrated that the gap between the model's current performance and theoretical ceiling is dominated by two structural gaps: (1) insufficient co-occurrence signal for tail items,

addressable with embeddings; and (2) price bias in the training data, addressable with segment-conditional features. Both P1 improvements together are estimated to deliver an additional +8–13pp combined Hit@8 improvement, potentially pushing Recall@8 above 80% and unlocking meaningful additional AOV lift.